

Step by Step Tutorial

Table of Contents

一步步真正写懂这份代码

结论

不要一上来复制全部文件。正确路线是每次只写一个模块，写完就问自己三个问题：输入是什么、输出是什么、出错时会怎样停下来。

第 0 步：先建 CubeMX 工程

目标：

- 能下载一个空工程到 STM32。
- 能让一个 LED 闪烁。

你需要确认：

- 使用的芯片型号。
- 按键 GPIO。
- LED GPIO 或 PWM。
- 电机驱动方式：PWM 还是 CAN。
- 编码器读取方式：TIM Encoder 还是外部中断。
- IMU 数据来源：SPI/I2C/UART/CAN。

没有这些信息也能先学软件结构，但不能说已经实车验证。

第 1 步：先写 pid.c

先理解 PID 的输入输出：

- 输入：目标值 target、反馈值 feedback、周期 dt_s

- 输出：控制量 output
- 核心误差： $\text{error} = \text{target} - \text{feedback}$

你要手写并测试：

```
PID_Init(&pid, 1.0f, 0.0f, 0.0f, -1000, 1000, -100, 100);  
out = PID_Calc(&pid, 100, 80, 0.005f);
```

如果 $k_p=1$ ，误差是 20，输出应接近 20。这个模块懂了，后面电机速度和云台角度都是同一种思想。

第 2 步：写 remote.c

先不要接复杂遥控器，只做按键：

- key1：切换底盘模式
- key2：yaw 目标角增加
- key3：pitch 目标角增加
- key_stop：急停/解除急停

关键不是按键本身，而是“消抖”和“短按/长按”。机械按键会抖动，如果不消抖，按一次可能被识别成多次。

你要能讲清楚：

- 为什么需要 REMOTE_DEBOUNCE_MS
- 为什么长按需要 REMOTE_LONG_PRESS_MS
- 为什么连发需要 REMOTE_REPEAT_MS

第 3 步：写 robot_def.h

这个文件只放公共定义：

- 整车模式
- 底盘模式
- 电机编号
- 限幅函数

- 关键参数宏

目的：让所有模块用同一套名字，避免到处出现魔法数字。

第 4 步：写 `chassis.c`

先只做运动学，不接电机：

```
w1 = +vx - vy - wz * K
w2 = +vx + vy + wz * K
w3 = -vx + vy - wz * K
w4 = -vx - vy + wz * K
```

你要用调试器观察：

- $v_x > 0$ 时四个轮子的目标值是否成对符合预期。
- $w_z > 0$ 时四个轮子是否形成旋转趋势。
- 输出是否被限幅。

然后再接 PID：

目标轮速 - 编码器反馈轮速 -> PID -> 电机输出

没有编码器时，`BSP_GetMotorSpeed()` 先返回 0，只能验证软件链路，不能说闭环实测完成。

第 5 步：写 `gimbal.c`

云台不要用编码器角度冒充 IMU。题目明确要求陀螺仪角度反馈。

先做目标角：

- key2 让 yaw 目标角每次加 5 度。
- key3 让 pitch 目标角每次加 3 度。
- pitch 目标角必须限制在 -20 到 30 度。

再做闭环：

目标角 - IMU 角度 -> PID -> 电机输出

如果没有 IMU，保留 `BSP_GetYawAngleDeg()` 和 `BSP_GetPitchAngleDeg()` 接口，并在 README 说明是待硬件接入。

第 6 步：写 `app.c`

`app.c` 是整个系统的节拍器：

```
Remote_Update();
Gimbal_Control();
Chassis_Control();
```

为什么先遥控，再云台，再底盘：

- 先读取输入，后续模块才有新命令。
- 云台先更新角度反馈，底盘跟随模式才能使用 yaw 信息。
- 底盘最后输出电机命令。

第 7 步：写 `bsp_port.c`

这里才接真实硬件：

- 按键：`HAL_GPIO_ReadPin`
- LED：`HAL_GPIO_WritePin` 或 PWM
- 电机：PWM 占空比或 CAN 电流
- 编码器：TIM Encoder 计数差分
- IMU：返回 yaw/pitch 角度

原则：硬件代码只放这里，不要散落到 `chassis.c` 和 `gimbal.c`。

第 8 步：理解 `safety.c`

安全模块不是为了多写一个文件，而是为了避免安全逻辑散落。

你要能讲清楚：

- `Safety_Update()` 每个周期收集当前安全状态。
- `Safety_ShouldStop()` 给调度层一个统一停机判断。

- `Safety_ReportMotorLimited()` 记录输出是否触碰限幅。

后续如果加入堵转、过温、低电压，也应该先进入 `safety.c`，再由 `app.c` 决定停机。

第 9 步：理解 `c620_can.c`

如果使用 RM 常见 C620 电调，需要知道两类 CAN 帧：

- 控制帧：主控发 4 路电流命令。
- 反馈帧：电调回传角度、转速、转矩电流、温度。

这个模块只负责“打包/解析”，不负责真正发送 CAN。真正的 `HAL_CAN_AddTxMessage()` 应该写在 `bsp_port.c` 或 CAN BSP 文件里。

第 10 步：逐项验证

不要直接上电让电机转。推荐顺序：

1. 只看 LED。
2. 只看按键状态。
3. 只看目标速度和目标角，不接电机。
4. 电机悬空，低限幅测试。
5. 接编码器反馈，看 PID 输出是否收敛。
6. 最后落地测试。

第 11 步：变成自己的代码

真正变成自己写的，不是改变量名，而是做到这三件事：

- 能画出模块关系图。
- 能说清楚每个函数的输入和输出。
- 能解释一次 bug 是怎么定位和修掉的。

建议他在 README 里写自己的调试记录，不要照抄模板。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:90